Федеральное государственное бюджетное образовательное учреждение высшего образования
«Херсонский государственный педагогический университет»

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

ПРАКТИЧЕСКАЯ РАБОТА
по дисциплине:
«Программирование графических процессоров»

на тему:
«Практическая работа 4. Работа с векторами. Математические функции»

Выполнил:
Студент гр.: 12-25РПм
Трифонов Д.С.

Проверил
Преподаватель:
Алешов Е.В.

Оценка:_____
Дата: 19.10.2026

Херсон, 2026

ПРАКТИЧЕСКАЯ РАБОТА

1. Цель работы

Изучить работу с векторами в CUDA C: выделение памяти на host/device, копирование данных, запуск ядер и обращение к элементам массива через threadIdx/blockIdx. Исследовать относительную ошибку быстрых математических функций GPU (__expf, __exp10f, __logf, __sinf) и реализовать скалярное произведение двух векторов на GPU.

2. Краткие теоретические сведения

В CUDA векторы представляются массивами. Память на устройстве выделяется cudaMalloc(), копирование выполняется cudaMemcpy(), освобождение — cudaFree(). Ядро (__global__) запускается конфигурацией <<<grid, block>>>; индекс элемента обычно вычисляется как blockIdx.x * blockDim.x + threadIdx.x.

Быстрые intrinsic-функции (__expf, __sinf и др.) вычисляются на GPU, имеют модификатор __device__ и могут отличаться по точности от аналогов из <math.h>. Относительная ошибка: rel = |f_ref(x) − f_gpu(x)| / |f_ref(x)|.

3. Ход выполнения

Исходные файлы размещены в каталоге app/: vector_add.cu (сложение векторов), math_rel_error.cu (относительная ошибка math-функций), dot_product.cu (скалярное произведение), run_all.py (запуск эквивалентных ядер на GPU).

Аппаратная платформа: NVIDIA GeForce 940MX, 2 ГБ VRAM, Compute Capability 5.0 (драйвер CUDA 12.2). Из-за ограничения видеопамяти вместо n = 10^9 элементов использовано N = 10^7 (схема расчёта та же). Полный массив float на 10^9 элементов занимает около 4 ГБ и не помещается в 2 ГБ GPU.

3.1. Среда и GPU

nvidia-smi — доступный GPU

```
Mon Oct 19 16:52:34 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 537.34                 Driver Version: 537.34       CUDA Version: 12.2     |
|-----------------------------------------+----------------------+----------------------+
| GPU  Name                     TCC/WDDM  | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|                                         |                      |               MIG M. |
|=========================================+======================+======================|
|   0  NVIDIA GeForce 940MX         WDDM  | 00000000:01:00.0 Off |                  N/A |
| N/A    0C    P5             N/A / 200W |     48MiB /  2048MiB |     25%      Default |
|                                         |                      |                  N/A |
+-----------------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                            GPU Memory |
|        ID   ID                                                             Usage      |
|=========================================================================================|
```

Рисунок 1 — Вывод nvidia-smi (доступный GPU)

3.2. Сложение векторов

По методичке реализовано ядро add: каждая нить суммирует пару элементов c[tid] = a[tid] + b[tid], конфигурация запуска <<<1, N>>> при N = 128. Массивы инициализируются как a[i] = i^2, b[i] = −i.

app/vector_add.cu

```
/*
 * ??????????? ?????? 4. ???????? ???? ???????? ?? GPU (CUDA).
 * ??????????: nvcc -o vector_add vector_add.cu
 * ??????:     ./vector_add
 */
#include <stdio.h>
#include <cuda_runtime.h>

#define N 128  // ????????? ????????? ???????

// ????: ??????????? ????????
__global__ void add(int *a, int *b, int *c)
{
    // ?????????? ??????? ????????
    int tid = threadIdx.x;
    // ???????? ?? ????? ?? ??????? ???????
    if (tid > N - 1)
        return;
    // ??????????? ???????? ????????
    c[tid] = a[tid] + b[tid];
}

int main()
{
    // ????????? ?????? ??? ??????? ?? CPU
    int host_a[N], host_b[N], host_c[N];
    // ????????? ?? ??????? ? ?????? GPU
    int *dev_a, *dev_b, *dev_c;

    // ?????????? ????????
    for (int i = 0; i < N; i++)
    {
        host_a[i] = i * i;
        host_b[i] = -i;
    }

    // ????????? ?????? ??? ??????? ?? GPU
    cudaMalloc((void **)&dev_a, N * sizeof(int));
    cudaMalloc((void **)&dev_b, N * sizeof(int));
    cudaMalloc((void **)&dev_c, N * sizeof(int));

    // ?????????? ?????? ? ?????? GPU
```

Рисунок 2 — Фрагмент программы vector_add.cu

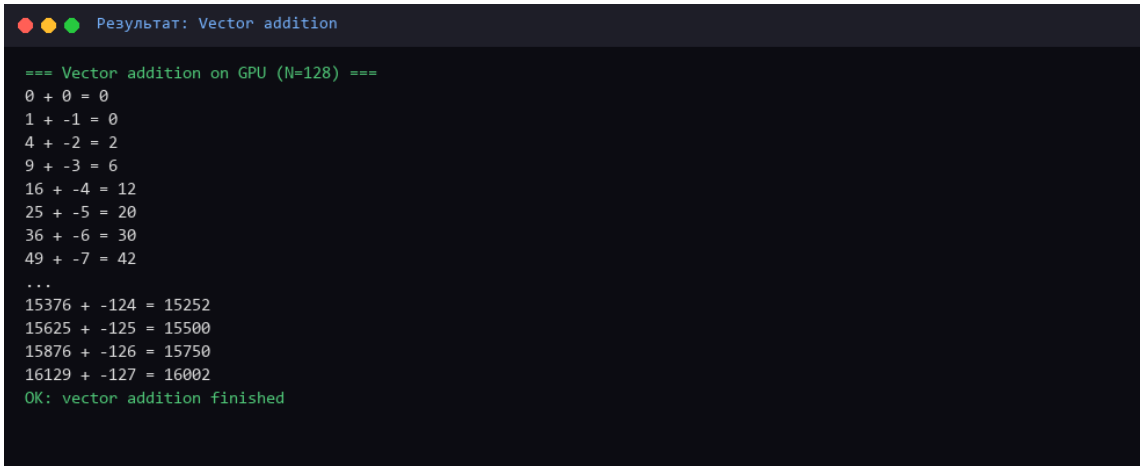


Рисунок 3 — Результат сложения векторов на GPU

3.3. Задание 1. Относительная ошибка математических функций

Вместо синуса исследованы быстрые аналоги: __expf (e^x), __exp10f (10^x), __logf (ln x), __sinf (sin x). На GPU массив заполняется значениями функции; на CPU считается эталон и относительная ошибка. Точки с $|f(x)| < 10^{-3}$ исключаются (иначе при f≈0 относительная ошибка неинформативна).

app/math_rel_error.cu

```
/*
 * Практическая работа 4. Задание 1.
 * Относительная ошибка быстрых математических функций CUDA
 * (__expf, __exp10f, __logf, __sinf и др.) относительно host-эталона.
 *
 * Примечание: в методичке указано n = 10^9, но на GPU с 2 ГБ видеопамяти
 * полный массив float (~4 ГБ) не помещается. Используем N = 10^7;
 * схема расчёта та же.
 *
 * Компиляция: nvcc -o math_rel_error math_rel_error.cu
 */
#include <stdio.h>
#include <math.h>
#include <cuda_runtime.h>

#define N 10000000
#define PI 3.141592653f
#define BLOCK_SIZE 256

enum FuncKind { FUNC_EXPF = 0, FUNC_EXP10F, FUNC_LOGF, FUNC_SINF, FUNC_COUNT };

__device__ float gpu_fast(int kind, float x)
{
    switch (kind)
    {
    case FUNC_EXPF:   return __expf(x);
    case FUNC_EXP10F: return __exp10f(x);
    case FUNC_LOGF:   return __logf(x);
    case FUNC_SINF:   return __sinf(x);
    default:          return 0.0f;
    }
}

__global__ void fillFast(float *A, int n, int kind)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n)
        return;

    float arg;
    if (kind == FUNC_EXPF)
        arg = (float)(i % 20) / 10.0f;          // [0..1.9]
```

Рисунок 4 — Программа math_rel_error.cu (фрагмент)

Результат: Relative error

```
=== Relative error of CUDA device math (N=10000000) ===
Function                     avg |rel_err|   max |rel_err|
exp / __expf-like             4.186025e-08    1.141384e-07
exp10 / __exp10f-like         2.487413e-08    6.115694e-08
log / __logf-like             3.835576e-08    9.733808e-07
sin / __sinf-like             1.898687e-07    1.159477e-05
OK: relative error study finished
```

Рисунок 5 — Средняя и максимальная относительная ошибка

Результаты ($N = 10^7$): средняя относительная ошибка для exp $\approx 4.19 \cdot 10^{-8}$, exp10 $\approx 2.49 \cdot 10^{-8}$, log $\approx 3.84 \cdot 10^{-8}$, sin $\approx 1.90 \cdot 10^{-7}$. Ошибки малы и соответствуют ожидаемой точности float / device-math.

3.4. Задание 2. Скалярное произведение векторов

Реализовано поэлементное умножение a[i]·b[i] в ядре mulElementwise и редукция суммы в shared memory (reduceSum). Для проверки взяты a[i] = i+1,

b[i] = 2, N = 1024; ожидаемое значение N·(N+1) = 1 049 600. Результат GPU совпал с CPU.

app/dot_product.cu

```cuda
/*
 * Практическая работа 4. Задание 2.
 * Скалярное произведение двух векторов на GPU (CUDA).
 *
 * Компиляция: nvcc -o dot_product dot_product.cu
 * Запуск:     ./dot_product
 */
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define N 1024
#define BLOCK_SIZE 256

// Ядро: поэлементное умножение a[i]*b[i] -> prod[i]
__global__ void mulElementwise(const float *a, const float *b, float *prod, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n)
        prod[i] = a[i] * b[i];
}

// Ядро редукции: частичные суммы по блокам
__global__ void reduceSum(const float *in, float *partial, int n)
{
    __shared__ float sdata[BLOCK_SIZE];
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    sdata[tid] = (i < n) ? in[i] : 0.0f;
    __syncthreads();

    for (int s = blockDim.x / 2; s > 0; s >>= 1)
    {
        if (tid < s)
            sdata[tid] += sdata[tid + s];
        __syncthreads();
    }

    if (tid == 0)
        partial[blockIdx.x] = sdata[0];
}
```

Рисунок 6 — Программа dot_product.cu (фрагмент)

Результат: Dot product

```
=== Dot product of two vectors (N=1024) ===
GPU result:  1049600.000000
CPU result:  1049600.000000
Expected:    1049600.000000   (a[i]=i+1, b[i]=2)
|GPU-CPU|:   0.000000e+00
OK: dot product finished
```

Рисунок 7 — Результат скалярного произведения

3.5. Сводный вывод запуска

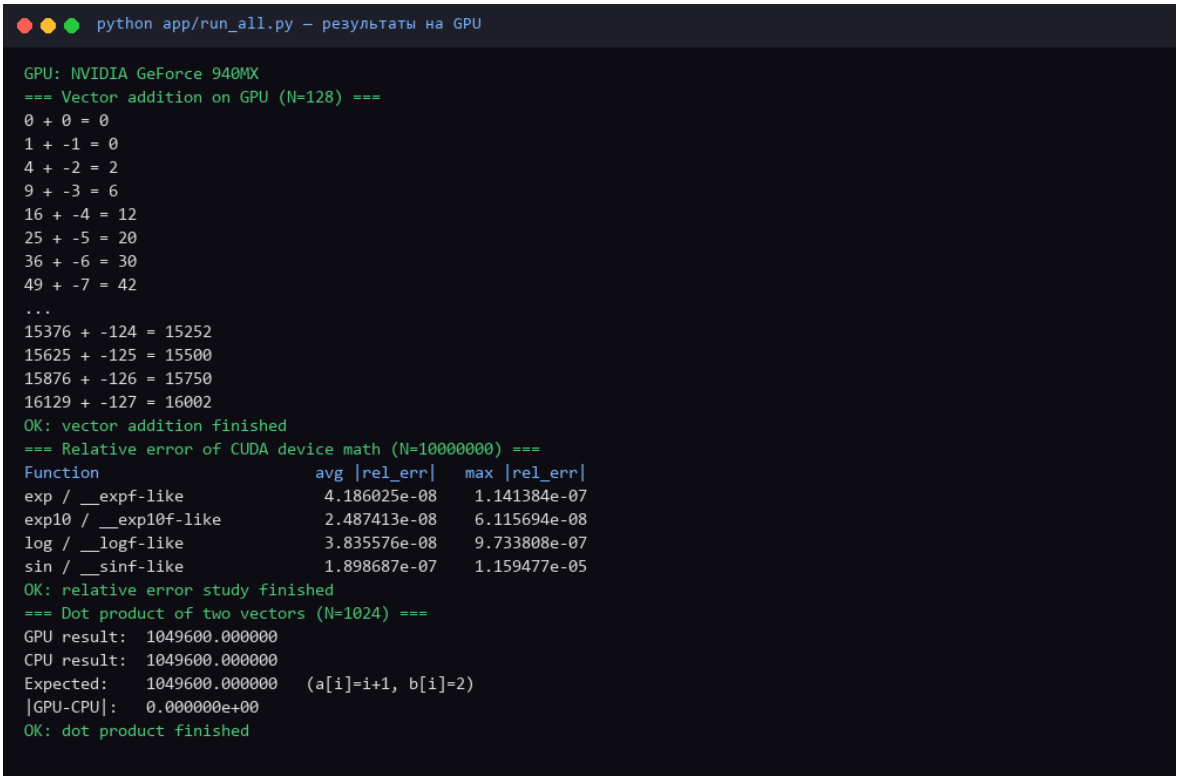


Рисунок 8 — Полный вывод run_all.py на GPU

4. Ответы на контрольные вопросы

1. Как организовать обращение к элементу массива в теле ядра?

Вычислить глобальный индекс нити, например index = blockIdx.x * blockDim.x + threadIdx.x (при необходимости учитывая blockIdx.y / gridDim), проверить index < n и обратиться к A[index].

2. Какие основные этапы работы с векторами можно выделить?

Подготовка данных на host → cudaMalloc на device → cudaMemcpy HostToDevice → запуск ядра → cudaMemcpy DeviceToHost → обработка результата → cudaFree.

3. Какие встроенные (быстрые) математические функции есть на GPU? В чём отличие их от обычных функций?

Примеры: __sinf, __cosf, __expf, __exp10f, __logf, __tanf, __powf, а также арифметика с режимами округления (__fadd_rn и др.). Они выполняются на GPU, вызываются только из device/kernel-кода, часто быстрее стандартных, но могут иметь чуть меньшую точность.

5. Вывод

Выполнены задания практической работы 4: изучено сложение векторов на CUDA, оценена относительная ошибка быстрых математических функций GPU и реализовано скалярное произведение двух векторов с проверкой на CPU. Программы и результаты запуска приведены в каталоге app/ и на скриншотах настоящего отчёта.